

Approximate Arithmetic Circuits Enabling Energy-Efficient Edge Computing

Salim Ullah 

Chair of Embedded Systems
Ruhr-Universität Bochum

Bochum, Germany

salim.ullah@ruhr-uni-bochum.de

Siva Satyendra Sahoo 

Technology Enablement and Design
Interuniversity Microelectronics Centre

Leuven, Belgium

Siva.Satyendra.Sahoo@imec.be

Akash Kumar 

Chair of Embedded Systems
Ruhr-Universität Bochum

Bochum, Germany

akash.kumar@ruhr-uni-bochum.de

Abstract—Approximate Computing (AxC) has emerged as a powerful technique for enabling efficient AI/ML workloads on resource-constrained edge devices. By judiciously introducing controlled inaccuracies, AxC leverages the inherent resilience of AI/ML models to unlock substantial gains in power, performance, and area (PPA). Nonetheless, the widespread adoption of AxC is hampered by the need for application-specific designs, which increase design and implementation complexity. To address this, we highlight advanced techniques for approximate arithmetic operator design—focusing on LUT-level optimizations, automated operator modeling, and design-space exploration on FPGAs. Through these methods, AI-enabled Electronic Design Automation (EDA) can effectively reduce development complexity and drive large-scale AxC adoption, paving the way for energy-efficient, scalable AI computing at the edge.

Index Terms—AI Computing, Arithmetic Circuits, Approximate Computing, Reconfigurable Architectures

I. INTRODUCTION

In recent years, the adoption of Artificial Intelligence (AI)-enabled processing at the edge has increased across various applications, such as autonomous vehicles, wearable health monitors, and industrial automation systems [1]–[3]. This shift allows for real-time decision-making, reduces latency, and minimizes reliance on cloud computing. However, these models' high computational complexity and memory footprint present significant challenges for real-time processing and energy consumption, especially for resource-constrained edge devices. For example, deploying a trained Convolutional Neural Networks (CNN) such as MobileNetV2 for image recognition on an edge device involves using 3.4 million parameters to perform approximately 300 million multiply-add operations per inference [4]. To this end, various software- and hardware-based optimization techniques have been proposed in the literature. Software-based techniques, such as quantization, pruning, and knowledge distillation, aim to reduce trained models' computational complexity and memory footprint by modifying their structure and precision. In contrast, hardware-based methods focus on reducing the computational cost per operation in Machine Learning (ML) models through specialized architectures and efficient processing units [5], [6].

Several hardware optimization methods have been proposed to reduce the cost of arithmetic operations involved in AI computing, encompassing both conventional approaches—such as quantization, truncation, and precision scaling—and more advanced techniques like Approximate Computing (AxC). In AxC controlled inaccuracies are introduced into computations in exchange for notable gains in Power-Performance-Area (PPA). While traditional methods primarily rely on scaling precision or pruning entire bits, logic pruning within Approximate Operator (AxO) targets the functional design itself, allowing for targeted removal of logic elements that minimally impact the final accuracy. However, the feasibility and benefits of such pruning can differ significantly across computing platforms. While logic pruning for Application-specific Integrated Circuit (ASIC) implementations could involve removing select gates from the accurate implementation

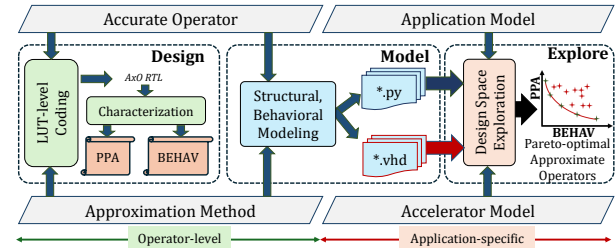


Figure 1: Methodology for implementing AxOs [7]

of an arithmetic operator, Field Programmable Gate Array (FPGA)-based implementations usually involve selective removal of Lookup Tables (LUTs) and Carry-Chains (CCs).

One of the primary benefits of designing an AxO over other hardware methods, such as precision scaling and quantization, is the flexibility to tailor optimizations to specific application requirements at a fine-grained level. Instead of uniformly reducing bit-width or pruning entire regions of logic, AxOs enable selective removal or alteration of logic elements that have minimal impact on the final output of a particular application. This approach dramatically expands the design space—each pruned gate or LUT combination can yield a unique performance, power, and accuracy trade-off—turning AxO design into a combinatorial search problem. To tackle this challenge, researchers are increasingly looking to intelligent Design Space Exploration (DSE) methods that leverage AI/ML techniques to guide the search toward optimal solutions. Recently, several methods have been proposed to leverage the growing AI/ML advancements into intelligent DSE methods. This ranges from using ML-based pseudo fitness functions to using more complex generative and explainable AI for aiding the design search algorithms.

As shown in Figure 1, the design and implementation of effective AxOs requires investigation across multiple aspects – approximate circuit design, modeling approximations, and enabling effective application-specific DSE [7]. To this end, the current article provides a detailed overview of such aspects and a brief survey of related works. The related contributions are:

- We present a description of different methods associated with the design and implementation of AxOs for FPGAs in Section II. Specifically, we present the specialized methods needed to ensure FPGA-specific optimizations that enable better PPA-error trade-offs compared to generic platform-agnostic approaches.
- We present operator modeling methods for generating application-specific AxO designs in Section III. Specifically, we describe how modeling the circuits, along with automated circuit generation, allows us to search for near-optimal designs for each application more effectively.
- We present an overview of both traditional and advanced DSE methods in use for exploring AxO designs. Specifically, we

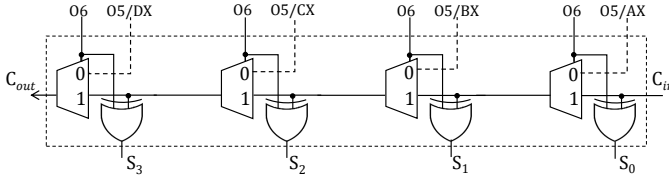


Figure 2: A 4-bit wide carry chain of AMD (Xilinx) FPGAs [9]: O6 (carry-propagate) and O5 (carry-generate) signals are generated by a 6-input LUT. The other inputs, such as AX and BX, are bypass signals.

describe how modern AI/ML approaches can be leveraged in the search for novel application-specific AxOs.

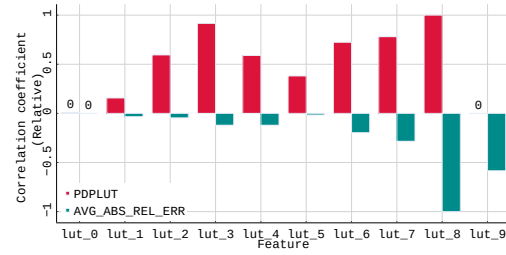
II. APPROXIMATE OPERATORS USING FPGAs

This section provides an overview of some of the recent works on the functional approximation of arithmetic operators, with a focus on adders and multipliers. It should be observed that approximation techniques developed for ASICs often yield limited or no performance improvements when directly implemented on FPGAs, and vice versa [8]. This limitation arises from the fundamental architectural differences between the two platforms. Modern FPGAs, such as those from AMD (Xilinx), rely on LUTs and carry chains (as shown in Figure 2) for combinational logic implementation [9]. Therefore, to achieve an optimal accuracy-performance trade-off, it is important to develop platform-aware approximation techniques that leverage the unique characteristics of each implementation.

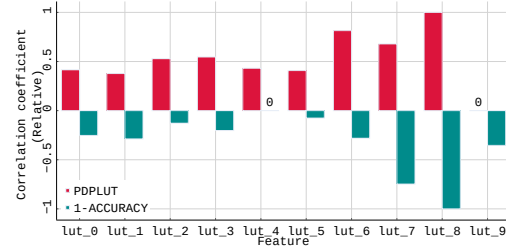
Functional approximation techniques intentionally introduce controlled output errors¹ to exchange output accuracy for improved performance. In this regard, circuit pruning to generate AxOs has been a primary method for introducing functional approximations. For example, the work presented in [10] presents various approximate adder designs by selectively pruning different transistors from the accurate circuit. Similarly, the framework presented in [11] enables gate-level pruning to create approximate adders that offer different accuracy-performance trade-offs. This framework in [11] utilizes a gate's significance score and toggle count as criteria to determine the order in which gates should be pruned from the accurate implementation. Among the various approximation techniques for adders, segmentation and carry chain prediction are extensively researched to enhance performance. For instance, the authors of [12] proposed a carry speculation method for ripple carry adders, which reduces critical path delay and improves efficiency. The authors of [13] proposed various approximate half and full adders for FPGA-based systems, utilizing LUTs of FPGAs to implement these different approximations. The authors of [14] have introduced a framework for designing approximate adders for Spiking Neural Networks (SNNs) by pruning LUTs from the initial implementation. In this approach, pruned LUT logic is replaced with a constant 1 or 0, generating a total of 3^n approximate circuits for an n -LUT-based design.

In comparison to addition, multiplication is computationally more complex, resulting in a wider array of approximation techniques designed to optimize speed, power, and area [8], [15]–[22]. For example, several studies have investigated truncation-based approaches to optimize $N \times N$ multipliers. Some methods focus on truncating the output to N bits [15], while others reduce the precision of the operands by using a smaller $N \times N$ multiplier to approximate

¹Other approximation methods, such as voltage over-scaling and over-clocking, result in non-deterministic errors due to timing violations.



(a)



(b)

Figure 3: Correlation of LUTs with accuracy (a) 4-bit signed multiplier (b) ECG classification using 4-bit signed multiplier [22]

$N \times N$ multiplications, where $M < N$ [23]. Other research efforts concentrate on approximating specific sub-operations within the multiplication process. For example, the authors of [16] introduce 2×2 approximate unsigned multiplier blocks for ASICs, utilizing simplified Karnaugh maps to reduce multiplication logic complexity. The 2×2 design is then used to implement higher-order (2×2 , 16×16) multipliers. The authors of [17] have presented a library of approximate multipliers and adders, integrating various previously proposed designs to provide designs with different accuracy-performance trade-offs.

Several works have focused on designing approximate multipliers for FPGA-based systems, exploring various techniques to enhance efficiency and reduce resource utilization. The work in [8] introduces 4×2 and 4×4 approximate unsigned multipliers, leveraging the truncation of least significant product bits and approximate addition of partial products to minimize LUT usage. Expanding on this, the authors of [18] have proposed a library of approximate multipliers by developing three distinct 4×4 approximate multipliers, where parallel and approximate computation of product bits eliminates carry propagation, further optimizing performance. Similarly, the authors of [19] have presented an approximate radix-4 Booth-based signed multiplier, truncating the least significant partial product bits to improve efficiency.

Most AxOs-related works follow an application-agnostic design approach, where AxOs are designed and optimized based on operator-level error and performance metrics. These operators are then tested across various applications to evaluate their effectiveness. However, this methodology has two key limitations:

- The application-agnostic design may lead to the creation of AxOs that do not fully satisfy the specific accuracy and performance needs of a given application.
- There is a lack of a structured methodology for generating new AxOs that offer different accuracy-performance trade-offs.

To address these issues, recent studies, such as [14], [20]–[22], [24], [25] have introduced generalized methodologies for developing

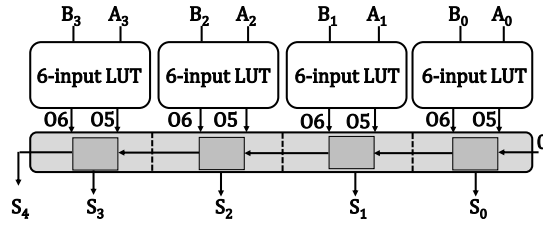


Figure 4: LUT mapping of a 4-bit accurate adder

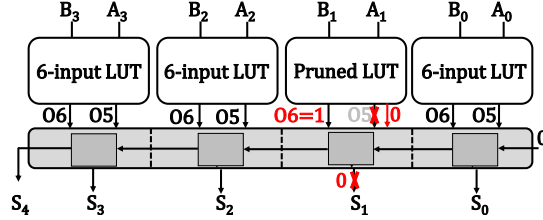


Figure 5: Approximate adder design according to input config '1101'

application-specific AxOs, ensuring that they better meet the requirements of target applications.

III. APPLICATION-SPECIFIC APPROXIMATE OPERATORS

Incorporating application-specific characteristics into AxOs design can lead to better accuracy-performance trade-offs at the application level. For example, the authors of [14] have implemented approximations to prevent overflows in the design of FPGA-based accumulators for SNNs. SNNs operate by accumulating spikes and generating an output spike once the accumulated value exceeds a threshold. However, traditional accurate adders are susceptible to overflow, which can degrade performance. In [14], deliberate approximations in the accumulation process prevent overflows while maintaining computational efficiency. Their proposed approach utilizes an n -bit approximate adder that requires the same hardware resources as an n -bit accurate adder, yet significantly improves application-level accuracy by ensuring reliable accumulation without overflow errors.

Application-agnostic pruning techniques, such as those in [19], optimize operators without considering application-level error and performance metrics. For example, Figure 3 presents an accuracy-performance correlation analysis for various 4×4 signed approximate multipliers, derived from the accurate signed multiplier in [8], which uses 10 LUTs for partial product generation. In Figure 3, different approximate circuits are created by systematically removing these LUTs. Figure 3 illustrates the correlation between each LUT's utilization and the PPA (Power $\times$ Delay $\times$ LUTs) and BEHAV (error) metrics. Retaining an LUT increases resource usage, critical path delay, and dynamic power consumption, whereas removing it introduces higher errors—manifesting as AVG_ABS_REL_ERR at the operator level and (1 - Accuracy) in ECG peak detection using a Low-pass Filter (LPF) [26]. The variation in correlation metric distribution between Figure 3(a) and Figure 3(b) reflects a shift from operator-level to application-level dependency. To address this, recent works, such as [20], [21] introduce selective LUT pruning techniques, allowing LUT removal to be guided by application-specific accuracy and performance constraints.

A. Selective LUT pruning

The AppAxO framework, proposed in [20], introduces a systematic approach for designing approximate arithmetic operators from their accurate counterparts. It leverages 6-input LUTs and carry chains

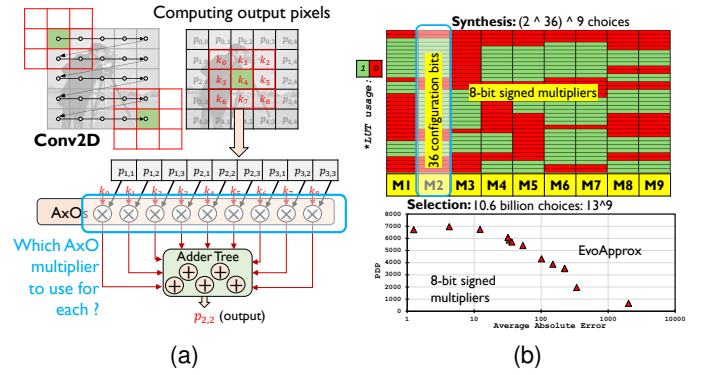


Figure 6: Selection vs Synthesis (a) 2D Convolution using approximate multipliers (b) Design space using *Synthesis* [20] and *Selection* [17] approaches

in FPGAs, applying approximations by selectively disabling LUTs and their associated carry chain cells. For an arithmetic operator employing K LUTs for logic implementation, AppAxO uses a K -bit configuration string, where a '0' in any position in the bit string disables the corresponding LUT. A disabled LUT does not contribute to intermediate results, and its *carry-propagate signal* (O6) is fixed to '1', while the *carry-generate signal* (O5) is externally bypassed and set to '0'. This setting ensures that the preceding carry propagates without generating a new one. Additionally, the *XOR gate* output from the carry chain cell is truncated to '0', allowing synthesis tools to completely omit disabled LUTs during implementation, optimizing resource utilization.

For example, Figure 4 illustrates the LUT-based implementation of an accurate unsigned 4-bit adder. The employed LUTs calculate the required *carry-generate* (O5) and *carry-propagate* (O6) signals by performing logical 'AND' and 'XOR' operations on the input bits, respectively. Applying the AppAxO methodology, a 4-bit configuration string is used to address each LUT within the accurate implementation. The 4-bit configuration string defines different approximate implementations. For example, Figure 5 depicts a 4-bit approximate adder with the configuration '1101'. In general, for an N -bit accurate adder, AppAxO generates 2^N approximate variants, each offering a unique accuracy-performance trade-off. Similarly, for the 4×4 multiplier analyzed in Figure 3, AppAxO produces 2^{10} (1,024) distinct approximate designs, enabling fine-grained optimization for different application requirements.

B. Selection vs. Synthesis

The application-specific AxO design, by synthesizing novel AxOs through selective LUT pruning, provides a fine-grained approach to extracting the maximum benefits for a given task. Rather than relying on a fixed library of approximate operators, the synthesis method systematically explores how minor logic-level modifications—reflected in the presence or absence of individual LUTs affect accuracy, power, and performance. For instance, consider a 3×3 convolutional filter composed of nine multipliers, as shown in Figure 6(a). As the output image is generated using a moving window of size 3×3 , each pixel in the window is multiplied with corresponding values in the filter kernel and then accumulated for generating each output pixel. Even small decisions to prune or retain each LUT can substantially alter an operator's computational characteristics, ultimately leading to an enormous number of design configurations. This combinatorial explosion underscores the inherent complexity of

the synthesis approach, as each pruning choice has a direct impact on the accuracy-power-performance trade-offs. For instance, assuming 8-bit signed multipliers, frameworks such as AppAxO [20], [21] can generate up to 2^{36} unique AxO implementations. When these decisions are multiplied across all nine operators in a filter, the design space becomes vast, highlighting the critical need for intelligent, automated methods to search for optimal solutions. For such a fine-grained optimization, the design space includes finding a unique AxO for each of the 9 multipliers, resulting in a total of $2^{36 \times 9}$ design points for the Conv2D accelerator. The design space using the synthesis approach thus grows exponentially, demanding advanced DSE techniques to uncover the “best fit” approximate operators for specific applications.

In contrast, some related works present a selection-driven approach to finding the *best-effort* application-specific AxOs by drawing from a predetermined library of approximate operators. Rather than pruning LUTs at a fine-grained level to synthesize entirely new designs, this method relies on comparing and evaluating the accuracy, power, and performance characteristics of pre-existing AxOs under real-world workloads. By focusing on a finite set of pre-designed operators, the selection process significantly reduces the overall design space compared to the combinatorial explosion seen in synthesis-based methods. For instance, as shown in Figure 6(b), if we consider the set of 13 unique 8-bit signed approximate multipliers from the EvoApprox library [17], the corresponding design space for the 3×3 Conv2D accelerator has 13^9 design points. The designer’s task then involves matching the application’s performance and accuracy requirements to the most suitable operator and this approach has been used in some related works [14], [19], [27]–[29]. While this streamlined process often lowers the complexity of design space exploration and shortens development time, it can also limit the degree of application-specific fine-tuning. Consequently, a designer working on highly specialized tasks may find that no existing AxO precisely meets their requirements, leading to potential suboptimal performance or higher energy consumption. Irrespective of the method—selection or synthesis—the design of application-specific AxOs requires maneuvering a large design space, underscoring the importance of intelligent exploration strategies and robust design tools.

IV. DSE FOR AXOS

A. Automated DSE

As mentioned earlier, selecting the right subset of LUTs to be pruned that provides the best accuracy-PPA tradeoff poses a combinatorial problem. In manual DSE, designers often identify critical LUTs based on their power dissipation and placement within the critical path, selectively pruning elements that minimize accuracy degradation while maximizing efficiency [19]. However, this approach becomes increasingly impractical as the design complexity scales. In the selection approach, the design space is constrained by the predefined set of available AxOs, limiting exploration but making evaluation more feasible. In contrast, the synthesis approach exponentially expands the design space, as each LUT in a synthesized operator can be independently pruned or retained, leading to an overwhelming number of potential configurations. This growth is particularly evident in fundamental arithmetic operations such as adders and multipliers, where each logic modification introduces a distinct accuracy-energy tradeoff. As illustrated in Figure 6, a 3×3 convolutional kernel implementing nine multipliers can be configured in over 10 billion ways using selection, whereas synthesis introduces a practically unbounded search space. Due to this complexity, researchers have explored various random sampling-based methods to

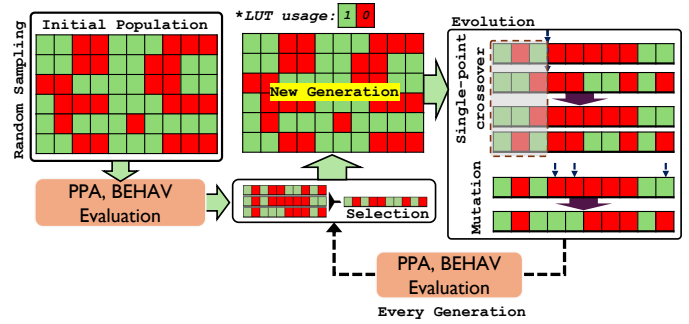


Figure 7: Genetic Algorithms-based AxO Synthesis

navigate the design space, including direct pruning strategies that iteratively refine a subset of candidate designs. However, leveraging advanced optimization methods requires representing designs in some standard structures, enabling automated search algorithms to systematically generate and evaluate approximate designs.

More intelligent search methods than random search have been used for AxO design and implementation, leveraging metaheuristic algorithms to efficiently explore the vast design space. One of the most widely used approaches is Genetic Algorithms (GA), which mimic natural evolution by iteratively refining a population of candidate designs based on selection, crossover, and mutation [20]. At the core of this process, each AxO design is encoded as a binary string, where each bit represents the presence or absence of a LUT within the FPGA-based structure. This encoding enables arbitrary instantiation and systematic random sampling, allowing for broad design exploration without the need for exhaustive enumeration. The GA-based search process follows several key stages:

- **Random Sampling (Initialization):** The first step involves generating an initial population of candidate designs, each representing a unique LUT pruning configuration. These designs are evaluated based on PPA, as well as accuracy metrics, to assess their suitability for approximation.
- **Selection:** Once the initial population is evaluated, the best-performing designs—those that strike a favorable balance between approximation error and hardware efficiency—are selected to form the basis of the next generation.
- **Crossover (Recombination):** The selected designs undergo crossover, where portions of their LUT configurations are combined to create new candidate designs. This step introduces diversity and helps discover novel combinations that may yield better results.
- **Mutation:** To further explore the design space and avoid premature convergence to suboptimal solutions, random mutations are introduced by flipping bits in the binary encoding. This simulates small, incremental changes to the LUT pruning patterns, allowing for a more refined search over multiple generations.

Figure 7 illustrates how an AxO’s binary representation evolves across generations. The process continues iteratively, with each new generation improving upon the previous one, until an optimal or near-optimal AxO configuration or a set of Pareto-optimal AxO configurations are found. Despite its effectiveness, a key bottleneck in the GA-based search is the need to evaluate the PPA and error characteristics of each design point. Since each configuration must undergo synthesis and simulation to obtain accurate performance metrics, the computational overhead can quickly become prohibitive, particularly for large-scale explorations. This highlights the need for

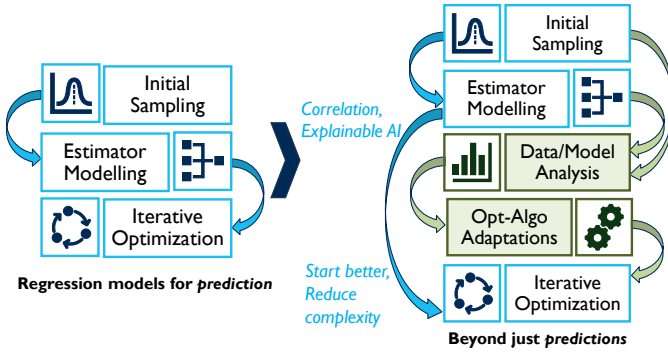


Figure 8: Using AI/ML for DSE of AxOs

efficient surrogate models or AI-driven estimators to accelerate the evaluation process.

B. AI/ML-enabled DSE

Recently, an increasing number of works have demonstrated the effectiveness of AI/ML methods in Electronic Design Automation (EDA). While AI has traditionally been used in specialized tasks such as IR drop analysis and placement optimization, it has now emerged as a powerful tool for addressing some of the challenges in DSE: navigating large, complex search spaces efficiently. Metaheuristic algorithms, such as GA and simulated annealing, require frequent evaluations of PPA as well as behavioral accuracy (BEHAV) metrics, making them computationally expensive. To mitigate this, AI/ML-based regression models have been introduced as an effective way to approximate fitness estimation, reducing the need for costly full-fledged synthesis and simulation at every design point.

Figure 8(left) illustrates this concept by depicting how regression models are integrated into the search process. The first step involves initial sampling, where a subset of the design space is evaluated using traditional methods to generate a training dataset for the regression model. Next, estimator modeling is performed, where AI/ML algorithms learn to predict PPA and BEHAV characteristics based on design parameters. This predictive capability enables faster iterative optimization, as search algorithms can now use estimated fitness values instead of running exhaustive simulations for each candidate solution. By leveraging regression models, designers can generate pseudo Pareto fronts—approximations of the optimal trade-off between PPA and accuracy—before refining them through selective verification on a smaller set of designs [20]. This drastically accelerates convergence in metaheuristic-based searches, allowing for a more efficient traversal of the design space while still maintaining high-quality solutions. However, most works using AI/ML for DSE have been limited to implementing regression-based fitness estimation only, leaving room for further advancements in leveraging AI for more adaptive and efficient DSE.

Recently, AI/ML algorithms have seen immense improvements, expanding beyond traditional regression models into more advanced paradigms such as Generative AI (GenAI), Explainable AI (xAI), and Large Language Model (LLM). These emerging techniques are beginning to transform the design of AxOs by enabling smarter, more adaptive, and data-driven design exploration strategies. While conventional AI-based approaches rely on regression models for estimating PPA and BEHAV metrics, next-generation AI-driven techniques aim to go beyond simple prediction and actively generate, interpret, and optimize AxO designs. One promising avenue is using GenAI techniques, such as Generative Adversarial Networks (GANs)

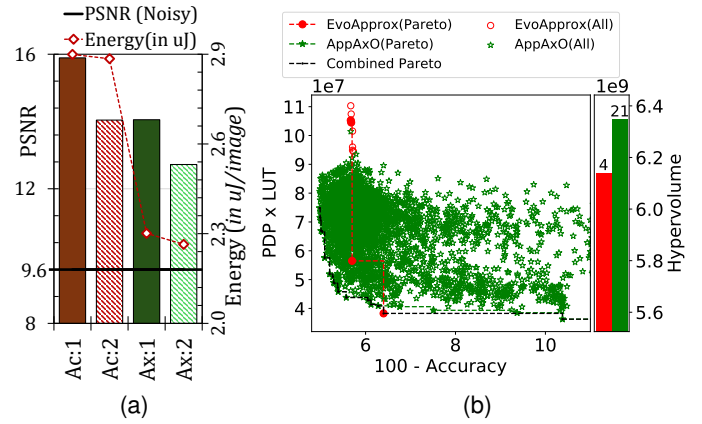


Figure 9: Energy efficiency using AxOs (a) Cross-layer approximation and energy for 2D convolution [29] (b) Energy(PDP)-LUT-Error trade-offs using AppAxO's automated AxO synthesis for MNIST digit recognition [20]

and supersampling methods, to synthesize new AxO designs. By training on existing high-performing AxOs, GANs can generate novel operator configurations that balance accuracy and efficiency while exploring regions of the design space that heuristic search methods might overlook [24]. Similarly, supersampling-based approaches can be used to interpolate and refine approximate designs, even predicting the appropriate AxO designs of higher bit-width from a set of lower bit-width designs [25]. In both [24], [25], the authors use the GenAI models to generate *one-shot* AxO designs for an application. Further, these models can also be used to generate initial solutions for iterative optimization methods such as GA.

Additionally, xAI techniques are increasingly being incorporated into the DSE process to interpret and refine AI-driven optimizations. Unlike black-box AI models, xAI augments DSE algorithms by providing insights into why certain design configurations perform well, enabling more structured decision-making. For instance, in [22], the authors have used the SHAP values from the regression models to predict the PPA and BEHAV metrics to guide a vanilla Monte Carlo Tree Search (MCTS) algorithm searching for novel AxOs. Similarly, in [30], the authors have used the regression models to formulate Mixed Integer Linear Programming (MILP) problems that augment the initial population of GA. The right half of Figure 8 illustrates this shift toward AI-augmented optimization strategies. It shows how model-driven analysis and correlation techniques can be used to refine optimization-algorithm adaptations, allowing AI models to actively adjust search strategies based on past evaluations. By leveraging model feedback, the optimization loop becomes more efficient, reducing redundant evaluations and prioritizing promising design regions.

V. CONCLUSION

Approximate Computing in general and approximate arithmetic operators, in particular, can provide disproportionate PPA-accuracy tradeoffs that emerging applications like AI computing can leverage effectively. In fact applications can benefit from multiple levels of approximation as shown in Figure 9(a). Using an AxO (Ax:1) in convolution operations can result in nearly 20% lower energy for achieving similar PSNR as using an algorithmic approximation method such as a larger stride length (Ac:2). However, finding the right application-

specific approximate configuration poses a significant challenge for AxO. Automated search methods form an integral component of AxO and AI/ML methods can be leveraged to extract significant benefits from application-specific AxO design. For instance, Figure 9(b) shows the Pareto-front that can be obtained from an application-specific synthesis of AxOs (AppAxO [20]) compared to that of a selection-based approach (EvOApprox [17]). The AppAxO approach uses ML-based fitness functions and results in AxO designs that provide much improved Energy(PDP)-LUT-Error trade-offs. This, along with using more advanced AI/ML methods can provide the path for widespread adoption of AxO in general and AxOs in particular.

ACKNOWLEDGEMENT

This work is supported by the Deutsche Forschungsgemeinschaft (DFG) under the X-ReAp project (Project number 380524764).

REFERENCES

- [1] M. R. Bachute and J. M. Subhedar, "Autonomous driving architectures: Insights of machine learning and deep learning algorithms," *Machine Learning with Applications*, vol. 6, p. 100164, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666827021000827>
- [2] S. Shajari, K. Kuruvinaiseti, A. Komeili, and U. Sundararaj, "The emergence of ai-based wearable sensors for digital health technology: A review," *Sensors*, vol. 23, no. 23, 2023. [Online]. Available: <https://www.mdpi.com/1424-8220/23/23/9498>
- [3] B. Maschler and M. Weyrich, "Deep transfer learning for industrial automation: A review and discussion of new techniques for data-driven machine learning," *IEEE Industrial Electronics Magazine*, vol. 15, no. 2, pp. 65–75, 2021.
- [4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.
- [5] A. Yazdanbakhsh, K. Seshadri, B. Akin, J. Laudon, and R. Narayanaswami, "An evaluation of edge TPU accelerators for convolutional neural networks," *CoRR*, vol. abs/2102.10423, 2021. [Online]. Available: <https://arxiv.org/abs/2102.10423>
- [6] T. Tan and G. Cao, "Deep learning video analytics through edge computing and neural processing units on mobile devices," *IEEE Transactions on Mobile Computing*, vol. 22, no. 3, pp. 1433–1448, 2023.
- [7] S. Ullah, S. S. Sahoo, and A. Kumar, "Enabling energy-efficient ai computing: Leveraging application-specific approximations : (education class)," in *2024 International Conference on Compilers, Architecture, and Synthesis for Embedded Systems (CASES)*, 2024, pp. 3–4.
- [8] S. Ullah, S. Rehman, M. Shafique, and A. Kumar, "High-performance accurate and approximate multipliers for fpga-based hardware accelerators," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2021.
- [9] Xilinx, "7 Series FPGAs Configurable Logic Block: User Guide," https://docs.xilinx.com/v/u/en-US/ug474_7Series_CLB, 2016.
- [10] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-power digital signal processing using approximate adders," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 1, pp. 124–137, 2013.
- [11] J. Schlachter, V. Camus, K. V. Palem, and C. Enz, "Design and applications of approximate circuits by gate-level pruning," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 5, pp. 1694–1702, 2017.
- [12] J. Hu and W. Qian, "A new approximate adder with low relative error and correct sign calculation," in *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2015, pp. 1449–1454.
- [13] B. S. Prabakaran, S. Rehman, M. A. Hanif, S. Ullah, G. Mazaheri, A. Kumar, and M. Shafique, "Demas: An efficient design methodology for building approximate adders for fpga-based systems," in *2018 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2018, pp. 917–920.
- [14] S. Ullah, S. S. Sahoo, and A. Kumar, "AxOSpike: Spiking Neural Networks-Driven Approximate Operator Design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 3324–3335, 2024.
- [15] H.-J. Ko and S.-F. Hsiao, "Design and application of faithfully rounded and truncated multipliers with combined deletion, reduction, truncation, and rounding," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 58, no. 5, pp. 304–308, 2011.
- [16] P. Kulkarni, P. Gupta, and M. Ercegovac, "Trading accuracy for power with an underdesigned multiplier architecture," in *2011 24th International Conference on VLSI Design*, 2011, pp. 346–351.
- [17] V. Mrazek, R. Hrbacek, Z. Vasicek, and L. Sekanina, "Evoapprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods," in *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2017, 2017, pp. 258–261.
- [18] S. Ullah, S. S. Murthy, and A. Kumar, "SMAproxlib: Library of FPGA-based approximate multipliers," in *Proceedings of the 55th Annual Design Automation Conference*, ser. DAC '18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: <https://doi.org/10.1145/3195970.3196115>
- [19] S. Ullah, H. Schmidl, S. S. Sahoo, S. Rehman, and A. Kumar, "Area-optimized accurate and approximate softcore signed multiplier architectures," *IEEE Transactions on Computers*, vol. 70, no. 3, pp. 384–392, 2021.
- [20] S. Ullah, S. S. Sahoo, N. Ahmed, D. Chaudhury, and A. Kumar, "AppAxO: Designing Application-specific Approximate Operators for FPGA-based Embedded Systems," *ACM Transactions on Embedded Computing Systems (TECS)*, 2022.
- [21] S. Ullah, S. S. Sahoo, and A. Kumar, "CoOAx: Correlation-aware Synthesis of FPGA-based Approximate Operators," in *Proceedings of the Great Lakes Symposium on VLSI 2023*, ser. GLSVLSI '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 671–677. [Online]. Available: <https://doi.org/10.1145/3583781.3590222>
- [22] S. S. Sahoo, S. Ullah, and A. Kumar, "AxOTreeS: A Tree Search Approach to Synthesizing FPGA-based Approximate Operators," *ACM Trans. Embed. Comput. Syst.*, vol. 22, no. 5s, Sep. 2023. [Online]. Available: <https://doi.org/10.1145/3609096>
- [23] S. Hashemi, R. I. Bahar, and S. Reda, "Drum: A dynamic range unbiased multiplier for approximate applications," in *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2015, pp. 418–425.
- [24] R. Ranjan, S. Ullah, S. S. Sahoo, and A. Kumar, "SyFAxO-GeN: Synthesizing FPGA-Based Approximate Operators with Generative Networks," in *Proceedings of the 28th Asia and South Pacific Design Automation Conference*, ser. ASPDAC '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 402–409. [Online]. Available: <https://doi.org/10.1145/3566097.3567891>
- [25] S. S. Sahoo, S. Ullah, S. Bhattacharjee, and A. Kumar, "AxOCS: Scaling FPGA-Based Approximate Operators Using Configuration Supersampling," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 6, pp. 2646–2659, 2024.
- [26] J. Pan and W. J. Tompkins, "A real-time qrs detection algorithm," *IEEE Transactions on Biomedical Engineering*, vol. BME-32, no. 3, pp. 230–236, 1985.
- [27] B. S. Prabakaran, V. Mrazek, Z. Vasicek, L. Sekanina, and M. Shafique, "Approxfpgas: Embracing asic-based approximate arithmetic components for fpga-based systems," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020.
- [28] V. Mrazek, M. A. Hanif, Z. Vasicek, L. Sekanina, and M. Shafique, "AutoAx: An Automatic Design Space Exploration and Circuit Building Methodology Utilizing Libraries of Approximate Components," in *Proceedings of the 56th Annual Design Automation Conference 2019*, ser. DAC '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: <https://doi.org/10.1145/3316781.3317781>
- [29] S. Ullah, S. S. Sahoo, and A. Kumar, "CLAppED: A Design Framework for Implementing Cross-Layer Approximation in FPGA-Based Embedded Systems," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE Press, 2021, p. 475–480. [Online]. Available: <https://doi.org/10.1109/DAC18074.2021.9586260>
- [30] S. S. Sahoo, S. Ullah, and A. Kumar, "AxOMaP: Designing FPGA-based Approximate Arithmetic Operators using Mathematical Programming," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 17, no. 2, Apr. 2024. [Online]. Available: <https://doi.org/10.1145/3648694>